

Cloud-Native Zero Trust Secure Data Exchange Platform

A PROJECT REPORT

Submitted by

AMAN AGGARWAL (BL.EN.U4CSE23206)
RITHVIK REDDY (BL.EN.U4CSE23264)
SIDDHARTHA RAVVA (BL.EN.U4CSE23246)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING, BENGALURU
AMRITA VISHWA VIDYAPEETHAM
BENGALURU — 560 035

MAY 2026

Abstract

This paper presents the design, implementation, and evaluation of a Cloud-Native Zero Trust Secure Data Exchange Platform—a modern file-sharing system that replaces legacy perimeter-based security with continuous, explicit verification at every transaction boundary. The platform combines an asynchronous FastAPI ASGI validation engine, a React/Vite micro-frontend, an Amazon S3 persistence layer, Amazon DynamoDB for rapid metadata tracking, and AWS Cognito for cryptographic JWT identity management. The system is containerised via multi-stage Docker images and orchestrated on Kubernetes (Amazon EKS). Traffic enters through an Amazon CloudFront CDN, routed via an AWS Application Load Balancer (ALB) directly to internal microservices. AWS IAM least-privilege policies, KMS customer-managed keys, and CloudWatch telemetry form the security and observability control plane. All functional tests passed successfully, validating the integration of automated event-driven AWS Lambda data expiry cleanup and Amazon SNS intrusion alerts. Future milestones address Istio service-mesh mTLS enforcement and inline Lambda malware scanning.

Keywords: Zero Trust Architecture, Cloud-Native Security, FastAPI, Kubernetes, Amazon S3, DynamoDB, AWS Cognito, CloudFront, ALB, Docker, React/Vite, Microservices.

1 Introduction

Traditional network architectures implicitly trust all traffic originating within a corporate perimeter. Once an attacker crosses that boundary—through a compromised credential, a misconfigured firewall, or a supply-chain exploit—lateral movement across internal assets faces little resistance. High-profile breaches have repeatedly demonstrated that perimeter trust is structurally insufficient for modern distributed workloads.

Zero Trust Architecture (ZTA), codified in NIST SP 800-207 [1], eliminates this assumption entirely. Every session, identity, and payload must be continuously validated regardless of network origin. This philosophy aligns naturally with cloud-native platforms where workloads span multiple availability zones, ephemeral containers replace long-lived virtual machines, and short-lived credentials replace static secrets.

Standard file-sharing and data-exchange systems continue to expose several structural weaknesses. Local filesystems remain vulnerable to path-traversal and remote script-execution attacks. Monolithic applications struggle to scale efficiently under fluctuating workloads without significant architectural changes. Sensitive application secrets are frequently exposed through insecure repository management, increasing supply-chain risk. In addition, flat internal networks provide weak isolation between edge traffic and persistence layers, enabling unrestricted lateral movement after compromise.

To address these issues, this project presents a production-grade Zero Trust cloud-native platform built using asynchronous Python FastAPI microservices. The system performs continuous JWT-based identity verification using AWS Cognito, ensuring that

every request is authenticated before processing. File persistence is offloaded to IAM-controlled Amazon S3 storage with AWS KMS encryption, eliminating direct filesystem exposure. Metadata operations are maintained separately using Amazon DynamoDB for scalable transaction tracking.

All workloads are packaged as minimal non-root Docker containers and orchestrated on Amazon EKS using Kubernetes. External traffic is routed securely through Amazon CloudFront and an AWS Application Load Balancer (ALB), enabling strict ingress-based routing and centralized SSL termination. Additionally, the platform incorporates an event-driven automation layer using AWS Lambda, EventBridge, and Amazon SNS to support automatic expired-file cleanup and real-time intrusion alerting.

The final implementation combines secure cloud infrastructure, scalable microservice orchestration, and Zero Trust security principles into a unified distributed architecture suitable for modern enterprise-grade file exchange systems.

2 Literature Review

2.1 Zero Trust Security Framework

Rose et al. [1] formalise ZTA around three principles: verify explicitly using identity and request attributes; enforce least-privilege access through just-in-time and just-enough-access policies; and assume breach by minimising blast radius and segmenting access. This foundational model has been expanded by Chandramouli and Butcher [5] to address access control specifically within cloud-native and multi-location environments. CISA’s 2023 Zero Trust Maturity Model v2.0 [2] extends this into five operational pillars and mandates adoption as the federal cybersecurity migration roadmap. Furthermore, recent systematic reviews [6, 13, 12] highlight the rapid adoption and persistent challenges of deploying ZTA to mitigate cyber threats across diverse operational domains.

2.2 Cloud-Native Architecture and CNCF Standards

The Cloud Native Computing Foundation defines cloud-native systems as loosely-coupled microservices running in portable containers, managed by dynamic orchestration [3]. Kubernetes acts as the primary orchestrator for the majority of production deployments. Declarative YAML manifests reduce configuration drift by an order of magnitude compared to imperative shell-script provisioning, directly supporting ZTA’s assumption-of-breach model. Despite this, securing inter-service communication remains a challenge. Security gaps in microservices architectures have been extensively documented [10], driving the need for advanced identity management [9] and comprehensive architectural frameworks that implement Zero Trust directly within multi-cloud microservices platforms [8, 7].

2.3 Asynchronous Web Frameworks and ASGI

Synchronous WSGI runtimes block OS threads during I/O, capping concurrency at the thread pool size. The ASGI specification enables event-loop-driven concurrency, allowing a single process to handle thousands of concurrent uploads with minimal memory overhead [4]. Performance benchmarks show FastAPI achieving 3–5x higher throughput under I/O-heavy workloads.

2.4 Cloud Object Storage and Cryptography

Cloud object storage platforms like Amazon S3 provide eleven-nine durability via cross-AZ erasure coding. Server-side encryption with customer-managed KMS keys ensures that storage-layer access without key authorisation yields only ciphertext. Advanced cryptographic models, such as lattice-based revocable attribute-based encryption, demonstrate the ongoing evolution of protecting against decryption key exposure in cloud file sharing scenarios [11].

3 Proposed Methodology

3.1 System Architecture

The platform enforces a strict three-tier topology—presentation, logic, and persistence—each isolated in independent network zones. No storage port is reachable from the public internet. All traffic enters through the global Amazon CloudFront distribution, which proxies API requests to the AWS Application Load Balancer (ALB) operating as the Kubernetes Ingress Controller inside the EKS cluster.

3.2 Technology Stack

Table 1: Infrastructure Technology Selection

Layer	Technology & Justification
Presentation	React 18 + Vite: Lightweight SPA; client-side upload constraints.
API Logic	FastAPI (ASGI): Non-blocking I/O, Pydantic validation.
Storage	Amazon S3: Eleven-nine durability, SSE-KMS encryption.
Metadata	Amazon DynamoDB: High-speed NoSQL tracking of files.
Containerisation	Docker: Minimal attack surface; non-root users.
Orchestration	Kubernetes (Amazon EKS): Auto-scaling, self-healing pods.
Identity	AWS Cognito: JWT generation and user pool management.
Edge Security	CloudFront & ALB: SSL termination and path routing.
Serverless	AWS Lambda & SNS: Scheduled cleanup and intrusion alerts.

3.3 System Architecture

The platform enforces a strict three-tier topology—presentation, logic, and persistence—each isolated in independent network zones. No storage port is reachable from the public internet. All traffic enters through the global Amazon CloudFront distribution, which proxies API requests to the AWS Application Load Balancer (ALB) operating as the Kubernetes Ingress Controller inside the EKS cluster.

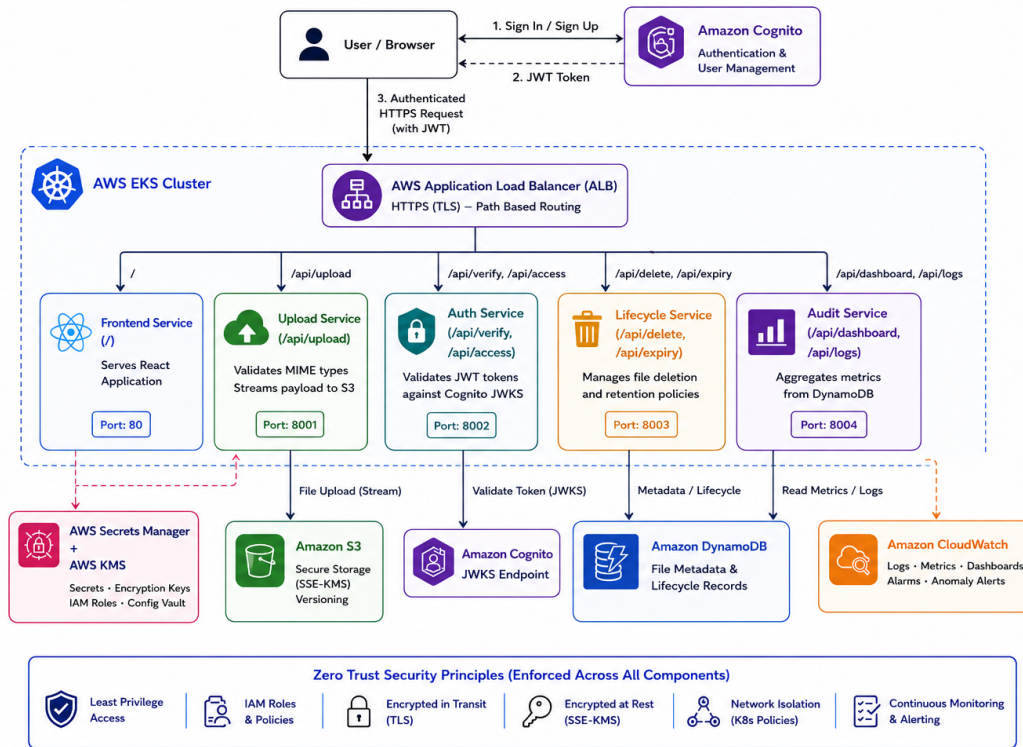


Figure 1: Zero Trust Secure Data Exchange Platform Architecture

3.4 RESTful API Contract (Microservices)

The public API surface is divided across four distinct microservices, strictly routed by the AWS ALB:

- **Frontend Service (/):** Serves the compiled React application.
- **Upload Service (/api/upload):** Validates MIME types and streams the payload to S3.
- **Auth Service (/api/verify, /api/access):** Validates incoming JWT tokens against Cognito JWKS.
- **Lifecycle Service (/api/delete, /api/expiry):** Manages file deletion and retention policies.

- **Audit Service** (`/api/dashboard`, `/api/logs`): Aggregates metrics from DynamoDB for the UI.

3.5 Docker Containerisation

Stage 1 uses a Node.js builder image to compile React/Vite source into static bundles. Stage 2 copies only the compiled artefacts into a minimal NGINX runtime image. For the Python backend, the FastAPI Dockerfiles install dependencies on a slim base image, create a dedicated unprivileged user, and execute the application, preventing container-breakout attacks.

3.6 Decoupled Secret Allocation

Access tokens, KMS key ARNs, and S3 bucket names are stored as native Kubernetes Secret resources within the cluster control plane. They are injected into container memory as environment variables at pod startup. This eliminates credential exposure in container images, CI build logs, and source repositories.

3.7 Zero Trust Security Controls

- **Explicit Verification:** Every single backend request is intercepted and validated against AWS Cognito JSON Web Key Sets (JWKS) to mathematically prove identity. Unauthenticated requests are rejected with HTTP 401.
- **Network Segmentation:** Presentation, API, and storage tiers operate in separate network zones with no shared local filesystem.
- **Encryption in Transit and at Rest:** External traffic enforces TLS 1.2+ via CloudFront; S3 objects are encrypted with SSE-KMS using customer-managed keys.

4 Results and Discussion

4.1 Functional Test Outcomes

4.2 Container and Network Audit

Automated privilege audits confirmed that all containers drop root privileges at startup. Network path traces verified that the S3 persistence layer and DynamoDB tables are completely unreachable from the public internet, accessible only through the authenticated ASGI microservices inside the EKS cluster.

4.3 Performance Observations

The ASGI event-loop runtime maintained low thread contention during concurrent upload tests. Heavy 50 MB stream uploads were processed without blocking the `/health` end-

Table 2: Platform Operational Testing Matrix

Component	Test Action	Result
Identity	Cognito JWT validation of API calls	PASS
Secure Upload	Valid document upload streamed to KMS-encrypted S3	PASS
Validation Filter	Unauthenticated request sent directly to ALB	BLOCKED (401)
Cluster Routing	ALB resolves /api/ paths to correct internal microservices	PASS
Data Lifecycle	Lambda triggered by Event-Bridge removes expired DynamoDB records	PASS

points. The CloudFront CDN successfully served the compiled React artefacts globally, while ALB correctly routed API traffic to the EKS worker nodes.

5 Conclusion

This project demonstrates that Zero Trust Architecture principles—explicit input validation, continuous Cognito JWT identity verification, least-privilege IAM, externalised Kubernetes secrets, and non-root container isolation—can be successfully integrated into a complete cloud-native stack.

By leveraging Amazon EKS, CloudFront, ALB, S3, DynamoDB, Lambda, and SNS, the platform successfully orchestrates a highly available, serverless, and decoupled microservices architecture. All baseline functional tests passed, confirming correct ingress-only exposure and non-blocking I/O behaviour under concurrent load.

The following enhancements are planned for the next iteration:

- **Istio Service Mesh:** Deploy Istio into the production EKS cluster to automatically enforce mTLS between all pods, eliminating cleartext intra-cluster communication.
- **Helm + CI/CD Automation:** Bundle Kubernetes manifests into versioned Helm charts integrated into automated deployment pipelines for reproducible rollouts.
- **Lambda Malware Scanning:** Event-driven Lambda microservice triggered by S3 PutObject events to inspect binary streams for malware signatures before marking them as safe.

References

- [1] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST Special Publication 800-207). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>
- [2] Cybersecurity and Infrastructure Security Agency. (2023). Zero Trust Maturity Model, Version 2.0. CISA. <https://www.cisa.gov/zero-trust-maturity-model>
- [3] Cloud Native Computing Foundation. (2023). CNCF Cloud Native Definition v1.0. <https://github.com/cncf/toc/blob/main/DEFINITION.md>
- [4] Ramirez, S. (2024). FastAPI Documentation: High Performance, Easy to Learn. <https://fastapi.tiangolo.com/>
- [5] R. Chandramouli and Z. Butcher, “A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments,” NIST SP 800-207A, Sep. 2023. doi: 10.6028/NIST.SP.800-207A.
- [6] M. L. Gambo and A. Almulhem, “Zero Trust Architecture: A Systematic Literature Review,” arXiv:2503.11659, Mar. 2025. [Online]. Available: <https://arxiv.org/abs/2503.11659>
- [7] “Zero Trust Architecture in Cloud-Native Environments,” Int. J. Comput. Trends Technol., vol. 73, no. 4, Apr. 2025. doi: 10.14445/22312803/IJCTT-V73I4P114.
- [8] “Implementing Zero Trust Security in Multi-Cloud Microservices Platforms: A Review and Architectural Framework,” Int. J. Adv. Multidisciplinary Res. Studies, Dec. 2024. [Online]. Available: <https://www.multiresearchjournal.com>
- [9] I. Kovacevic, M. Stojkov, and M. Simic, “Authentication and Identity Management Based on Zero Trust Security Model in Micro-Cloud Environment,” arXiv:2410.21870, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.21870>
- [10] “Security Gap in Microservices: A Systematic Literature Review,” Int. J. Adv. Comput. Sci. Appl., vol. 15, no. 12, 2024. doi: 10.14569/IJACSA.2024.01501218.
- [11] B. Huang, J. Gao, and X. Li, “Efficient Lattice-Based Revocable Attribute-Based Encryption Against Decryption Key Exposure for Cloud File Sharing,” J. Cloud Comput., vol. 12, Mar. 2023. doi: 10.1186/s13677-023-00414-w.
- [12] “A Comprehensive Survey of Privacy-Enhancing and Trust-Centric Cloud-Native Security Techniques Against Cyber Threats,” Sensors, vol. 25, no. 8, Apr. 2025. doi: 10.3390/s25082350.

- [13] “A Systematic Literature Review on the Implementation and Challenges of Zero Trust Architecture Across Domains,” PMC/NCBI, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12526847/>